

Entorno Linux de la base de datos: configuración y registro de lo hecho

Equipo: Server-E (equipo de William Ortiz). **Fecha:** 4 de octubre de 2026.

Para qué sirve: reproducir en otro Linux todo lo que se configuró para la base de datos de Territorio Preparado, saber dónde quedó cada cosa y resolver los errores que ya aparecieron.

Este documento no contiene contraseñas, tokens ni claves.

1. El equipo y las herramientas

Herramienta	Versión usada	Para qué
Sistema operativo	Pop!_OS 24.04 LTS (kernel 6.18)	Base del entorno; sirve igual en Ubuntu 24.04
Node.js (con nvm)	24.21.0 (npm 11.19.0)	Pruebas, carga de datos y CLI de Supabase. Mínimo recomendado: Node 20
Git	2.43.0	Control de versiones
GitHub CLI (gh)	2.101.0	Pull requests, invitaciones y visibilidad del repo
CLI de Supabase	2.119.0	Enlazar el proyecto y aplicar migraciones. Instalada dentro de db/ , no global
PGlite + PostGIS	0.5.8 y 0.2.8 (PostgreSQL 18 + PostGIS 3.6 en WASM)	Probar el SQL sin Docker ni Supabase
pg (Node)	8.23.1	Conexión a Supabase para la carga de datos
OpenSSL	3.0.13	Revisar el certificado TLS de Supabase
Python	3.12.3	Conversor de Markdown a HTML (docs/md2html.py)
LibreOffice	24.2.7	HTML a PDF
Poppler (pdftopdf, pdftoppm)	24.02	Revisar los PDF generados

No se usa: Docker, la CLI global de Supabase ni PostgreSQL instalado en el sistema. Para PostgreSQL, el servidor remoto de Supabase es la versión 17; las pruebas locales usan la 18.

Instalación en un Linux nuevo (Ubuntu o Pop!_OS)

```
sudo apt update
sudo apt install -y git gh openssl python3 libreoffice-writer poppler-utils
# Node.js con nvm (sin sudo)
```

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
source ~/.bashrc
nvm install 24
# GitHub
gh auth login
```

2. Dónde quedó cada cosa

Ruta	Qué es	¿En el repo?
~/hackathon-cali-2026/	Clon del repo leonidas452528/hackaton-camaracomercio2026 (privado)	—
db/supabase/migrations/	Las 12 migraciones SQL	Sí
db/supabase/config.toml	Configuración de la CLI: solo el esquema api, hook del token, MFA TOTP, sin seed	Sí
db/supabase/.temp/	Lo que deja supabase link: project ref, URL del pooler (sin contraseña), versiones	No (ignorado)
db/certs/supabase-prod-ca-2021.crt	CA raíz pública de Supabase, para verificar TLS	Sí (no es un secreto)
db/node_modules/	Dependencias: PGLite, PostGIS, pg y la CLI de Supabase	No (se instala con npm install)
db/package.json	Scripts test, test:sql, test:carga y cargar	Sí
db/tests/	Imitación mínima de Supabase, ejecutor y pruebas	Sí
db/scripts/cargar_datos_reales.mjs	Carga de los datos reales	Sí
Token de la CLI de Supabase	Lo guarda supabase login en el llavero del sistema (GNOME Keyring), no en un archivo	No
Contraseña de la base de datos	Solo en el gestor de contraseñas de William	No
~/.config/territorio-preparado/	Contraseñas de las 13 cuentas de demostración (permisos 600)	No
maqueta3d/.env.local	URL y clave pública de Supabase para el frontend (cada integrante crea la suya)	No (ignorado)

3. Registro de lo hecho, en orden

3.1 Pruebas locales del SQL (sin Supabase)

PGLite corre PostgreSQL con PostGIS dentro de Node, porque en el equipo no hay Docker.

```
cd ~/hackathon-cali-2026/db
npm install          # PGLite, PostGIS, pg y la CLI de Supabase
npm test            # 69 pruebas del SQL + 17 de la carga de datos reales
```

3.2 Repositorio privado y equipo

```
# (el repo se pasó a privado desde la web: Settings → General → Danger Zone)
gh repo view leonidas452528/hackaton-camaracomercio2026 --json visibility
gh api -X PUT repos/leonidas452528/hackaton-camaracomercio2026/collaborators/helynecheverry -f
permission=push
gh api -X PUT repos/leonidas452528/hackaton-camaracomercio2026/collaborators/PabloEArangoM -f
permission=push
gh api -X PUT repos/leonidas452528/hackaton-camaracomercio2026/collaborators/bryanmavi -f
permission=push
git push -u origin db/esquema-inicial
gh pr create --base main --head db/esquema-inicial      # PR #2
```

3.3 CLI de Supabase dentro del proyecto

```
cd ~/hackathon-cali-2026/db
npm install -D supabase          # queda en db/node_modules/.bin/supabase
npx supabase init --with-vscode-settings=false --with-intellij-settings=false
```

Después se editó db/supabase/config.toml: project_id = "territorio-preparado", schemas = ["api"], hook pg-functions://postgres/idn/custom_access_token_hook, MFA TOTP activado y [db.seed] enabled = false.

3.4 Login y enlace (en una terminal NORMAL)

```
cd ~/hackathon-cali-2026/db
npx supabase login          # abre el navegador; cuenta de GitHub
npx supabase link --project-ref rqxltixtsiqsaxpdue      # pide la contraseña de la base
```

3.5 Aplicar las migraciones

```
npx supabase db push --dry-run      # mostró exactamente las 12 migraciones
npx supabase db push                # aplicó las 12 sin errores
npx supabase migration list         # lo local y lo remoto coinciden
```

3.6 Exponer solo el esquema api (en el panel)

Project Settings → *Data API* → *Exposed schemas*: solo api. Verificación con la clave pública:

```
curl -s -H "apikey: CLAVE_PUBLICA" -H "Accept-Profile: api" \
  "https://rqxltixtsiqsaxpdue.supabase.co/rest/v1/amenazas?select=codigo"
```

3.7 Certificado TLS de Supabase

```
cd ~/hackathon-cali-2026/db/certs
curl -sSf -o supabase-prod-ca-2021.crt \
  https://supabase-downloads.s3-ap-southeast-1.amazonaws.com/prod/ssl/prod-ca-2021.crt
openssl x509 -in supabase-prod-ca-2021.crt -noout -subject -dates -fingerprint -sha256
```

```
# Comprobar que el pooler presenta un certificado válido firmado por esa CA
echo | openssl s_client -starttls postgres -connect aws-0-us-east-1.pooler.supabase.com:5432 \
  -CAfile supabase-prod-ca-2021.crt -verify_hostname aws-0-us-east-1.pooler.supabase.com | grep "Verify
return code"
```

Resultado: CA "Supabase Root 2021 CA", vigente hasta el 26 de abril de 2031, y Verify return code: 0 (ok).

3.8 Carga de los datos reales

```
cd ~/hackathon-cali-2026/db
read -rs PGPASSWORD && export PGPASSWORD          # la contraseña se escribe oculta
DATABASE_URL="postgresql://postgres.rqxltixtsiqsaxapdue@aws-0-us-east-1.pooler.supabase.com:5432/
postgres?sslmode=verify-full&sslrootcert=$PWD/certs/supabase-prod-ca-2021.crt" npm run cargar
unset PGPASSWORD
```

Resultado: 9 huellas verificadas; 22 comunas, 342 barrios, 2.991 espacios, 660 zonas de amenaza y 182 JAC; cruces idénticos a los de la app (884 de inundación y 1.369 sísmicos).

3.9 Documentos en PDF

```
cd ~/hackathon-cali-2026/db/docs
python3 ../../docs/md2html.py ACCESO_EQUIPO.md /tmp/ACCESO_EQUIPO.html
soffice --headless --convert-to pdf:writer_web_pdf_Export --outdir . /tmp/ACCESO_EQUIPO.html
```

Igual para INFORME_MONTAJE_SUPABASE.md, este documento y la bitácora (docs/PROYECTO.md).

3.10 Cuentas de demostración (Hito 5)

```
cd ~/hackathon-cali-2026/db
npx supabase db push          # migración 13 (provisión por servidor)
export SUPABASE_URL=https://rqxltixtsiqsaxapdue.supabase.co
read -rs SUPABASE_SERVICE_ROLE_KEY && export SUPABASE_SERVICE_ROLE_KEY  # Project Settings → API Keys
                                   (legacy service_role)
read -rs SUPABASE_ANON_KEY && export SUPABASE_ANON_KEY                  # legacy anon
npm run cuentas-demo          # crea lo que falte y verifica las 13 cuentas
unset SUPABASE_SERVICE_ROLE_KEY SUPABASE_ANON_KEY
```

Las contraseñas quedan en ~/.config/territorio-preparado/cuentas_demo_rqxltixtsiqsaxapdue.csv (carpeta 700, archivo 600). Para cambiarlas todas: npm run cuentas-demo -- --rotar.

4. Errores que ya aparecieron y su solución

Error	Causa	Solución
LoginMissingTokenError: Cannot use automatic login flow inside non-TTY environments	supabase login se corrió dentro de un asistente, sin terminal interactiva	Correrlo en una terminal normal, o usar --token con un token de <i>Account</i> → <i>Access Tokens</i>
getaddrinfo EAI_AGAIN HOST	Se dejó el marcador HOST en la URL	Poner el host real del <i>Session pooler</i> : aws-

SELF_SIGNED_CERT_IN_CHAIN	El sistema no conoce la CA de Supabase	Agregar &sslrootcert=.../db/certs/supabase-prod-ca-2021.crt; nunca desactivar la verificación
Advertencia de pg: sslmode require ... treated as verify-full	Cambio anunciado de la librería	Usar sslmode=verify-full explícito
PGRST106 Invalid schema: api	El esquema api no estaba expuesto	<i>Project Settings</i> → <i>Data API</i> → <i>Exposed schemas</i> = api
password authentication failed	Contraseña incorrecta o ya cambiada	Restablecerla en <i>Project Settings</i> → <i>Database</i> y repetir supabase link
supabase db push pide la contraseña	El enlace guarda el proyecto, no la contraseña	Escribirla en el prompt de la CLI, o export SUPABASE_DB_PASSWORD solo en esa sesión

5. Higiene de secretos

- La contraseña nunca va dentro de la URL: va en PGPASSWORD, escrita con read -rs.
- Si una contraseña se escribe por error en la terminal, se borra del historial:

```
history | grep -n "texto"
history -d NUMERO && history -w
```

- **Pendiente:** la contraseña de la base del 4 de octubre quedó expuesta en el chat. Hay que restablecerla en *Project Settings* → *Database* → *Reset database password* y volver a hacer npx supabase link.
- La clave service_role o secret nunca va al navegador, al repo ni al chat. El frontend solo usa la clave **publishable**.

6. Comandos de uso diario

Tarea	Comando (desde ~/hackathon-cali-2026/db)
Probar todo localmente	npm test
Solo las pruebas del SQL	npm run test:sql
Solo la carga (en memoria)	npm run test:carga
Crear una migración	npx supabase migration new nombre_del_cambio
Ver qué falta aplicar	npx supabase db push --dry-run
Aplicar migraciones (solo el responsable)	npx supabase db push
Comparar lo local con el remoto	npx supabase migration list

Recargar los datos reales

Sección 3.8 (es idempotente)

Crear o verificar las cuentas de
demostración

Sección 3.10 (`npm run cuentas-demo`)